

Special Participation B: Coding Problem in HW 6 using Cursor

5. Zachary's Karate Club

Zachary's karate club (ZKC) is a social network of a university karate club, described in the paper "An Information Flow Model for Conflict and Fission in Small Groups" by Wayne W. Zachary.

The social network captures 34 members of a karate club, documenting links between pairs of members who interacted outside the club.

During the study, a conflict arose between the officer/ administrator ("John A") and the instructor "Mr. Hi", which led to the split of the club into two.

Part of the members formed a new club around Mr. Hi; and the remaining members went with the officer.

5(a)

Go through q_zkc.ipynb. We want our network to be aware of information about the nodes themselves instead of only the neighborhood, so we add self loops our adjacency matrix. The paper called this $\tilde{A}$. Compute $\tilde{A}$ to add self loops to your adjacency matrix.

input: complete Q1, calculate the A_{tild} following the reference

Cursor output:

```
# TODO
# A_tild = ???
A_tild = A + np.eye(A.shape[0])
```

```
A_tild = A + np.eye(num_nodes)
A_tild
```

Solution:

The answer is correct. The self-loops have been added to the adjacency matrix as expected.

5(b)

Write a function that takes in $\tilde{A}$ as argument and returns the $\tilde{A}^{\text{SymNorm}}$ adjacency matrix. You may find the provided function `fill_diagonal` useful, as well as the inverse function `np.linalg.inv` and the matrix square root function `np.linalg.sqrtm`.

input: complete Q2, write the code of function `get_adjacency_matrix` following the reference

Cursor output:

```
def get_adjacency_matrix(A_tild):
    # 计算度矩阵 D: 对角线元素是每个节点的度
    degrees = np.sum(A_tild, axis=1)
    D = fill_diagonal(np.zeros_like(A_tild), degrees)

    # 计算  $D^{-1/2} = (D^{1/2})^{-1}$ 
    D_sqrt = sqrtm(D)
    D_inv_sqrt = np.linalg.inv(D_sqrt)

    # 计算对称归一化的邻接矩阵:  $D^{-1/2} * A_{tild} * D^{-1/2}$ 
    A_sym_norm = D_inv_sqrt @ A_tild @ D_inv_sqrt

    return A_sym_norm
```

Solution:

```
def get_adjacency_matrix(A_tild):
    D_tild = np.zeros_like(A_tild)
    num_con_per_node = np.asarray(np.sum(A_tild, axis=1)).flatten()
    D_tild_flattened = fill_diagonal(D_tild, num_con_per_node)
    inv_of_root_of_D_tild = np.linalg.inv(sqrtm(D_tild_flattened))
    return inv_of_root_of_D_tild @ A_tild @ inv_of_root_of_D_tild
```

We run the function `get_adjacency_matrix` with the provided input and obtain the expected output for the symmetric normalized adjacency matrix. Cursor successfully indicates the formula: $D^{-1/2} * \tilde{A} * D^{-1/2}$

5(c)

The other input to our GNN is the graph node matrix X which contains node features. For simplicity, we set X to be the identity matrix because we don't have any node features in this example. Generate the feature input matrix X .

input: complete Q3, generate the feature input matrix X following the reference

Cursor output:

```
# TODO
# X = ??
X = np.eye(num_nodes)
```

Solution:

```
X = np.eye(num_nodes)
X
```

The feature input matrix X has been generated correctly as the identity matrix of size 34x34, which corresponds to the number of nodes in the Zachary's Karate Club graph.

5(d)

We will now implement a single layer GNN. Implement the forward and backward pass functions for `GNN_Layer` class. Details can be found in the notebook

input: complete Q4, implement the forward and backward pass functions for `GNN_Layer` class following the reference. Don't delete the provided code comments.

Cursor output:

```
def forward_pass(self, A, X, W=None):
    """A here is the symmetrically normalized adjacency matrix
    and X is the input to the layer. We cached some values
    to use in the backward pass."""

    self.cache['A'] = A # (batch_size, batch_size)
    # TODO
    # self.cache['X'] = ?? # (input_node_feature_dim, batch_size)
    self.cache['X'] = X.T # (input_node_feature_dim, batch_size)

    # if W is None:
    #     W = ??

    if W is None:
        W = self.W

    # H = ?? (hidden_dim, batch_size)
    H = W @ (A @ X).T # (output_dim, batch_size)
    # H = ?? (apply activation)
    # self.cache['H'] = ?? # (hidden_dim, batch_size)
    return # (batch_size, hidden_dim)
    H = self.activation(H) # (output_dim, batch_size)
    self.cache['H'] = H # (hidden_dim, batch_size)
    return H.T # (batch_size, hidden_dim)
```

```
def backward_pass(self, optimizer, need_update=True):
    # dtanh = ?? # (batch_size, output_dim)
    # optimizer.output contains the gradient from the next layer. It is multiplied by the batch size,
    # so you'll need to divide by optimizer.batch_size to get the average gradient.
    d = np.multiply(optimizer.output, dtanh) # (batch_size, output_dim)
    H_T = self.cache['H'].T # (batch_size, output_dim)
    dtanh = 1 - H_T ** 2 # (batch_size, output_dim) - derivative of tanh
    d = np.multiply(optimizer.output / optimizer.batch_size, dtanh) # (batch_size, output_dim)

    # self.grad = ?? (batch_size, input_dim)
    self.grad = self.cache['A'].T @ d @ self.W.T # (batch_size, input_dim)

    optimizer.output = self.grad

    # dW = ?? # (output_dim, input_node_feature_dim)
    AX = self.cache['A'] @ self.cache['X'].T # (batch_size, input_dim)
    dW = d.T @ AX / optimizer.batch_size # (output_dim, input_dim)
    # dW_weight_decay = ??
    dW_weight_decay = self.W * optimizer.weight_decay / optimizer.batch_size

    # if need_update: # Use the gradient descent update rule on W. Remember to include weight decay.
    #     self.W = ??

    if need_update:
        self.W = self.W - (dW + dW_weight_decay) * optimizer.learning_rate

    return # (output_dim, input_node_feature_dim)
    return dW + dW_weight_decay # (output_dim, input_node_feature_dim)
```

Python

Solution:

```
def forward_pass(self, A, X, W=None):
    """A here is the symmetrically normalized adjacency matrix
    and X is the input to the layer. We cached some to the value
    for the case where we will need them later."""

    self.cache['A'] = A
    self.cache['X'] = (self.cache['A'] @ X).T

    if W is None:
        W = self.W

    H = W @ self.cache['X']
    H = self.activation(H)
    self.cache['H'] = H
    return self.cache['H'].T

def backward_pass(self, optimizer, need_update=True):
    dtanh = 1 - np.asarray(self.cache['H'].T)**2
    d = np.multiply(optimizer.output, dtanh)

    self.grad = self.cache['A'] @ d @ self.W
    optimizer.output = self.grad

    dW = np.asarray(d.T @ self.cache['X'].T) / optimizer.batch_size
    dW_weight_decay = self.W * optimizer.weight_decay / optimizer.
    batch_size

    if need_update: # Use the gradient descent update rule on W.
        Remember to include weight decay.
        self.W = self.W - (dW + dW_weight_decay) * optimizer.
        learning_rate

    return dW + dW_weight_decay
```

Python

let's check the result in 5(e)

5(e)

Run the forward and backward passes and ensure the checks pass, compute the forward passes, uncomment and complete.

```
# TODO
gnn_layer_output = gnn_layer.forward_pass(A=A_hat,
                                          X=X)

optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)
```

✓ 0.0s

Python

```
dW_approx = compute_gradients(param_name="W",
                              layer=sm_layer,
                              inputs_data=(gnn_layer_output,),
                              gt_labels=labels,
                              eps=1e-4,
                              weight_decay=optim.weight_decay)

db_approx = compute_gradients(param_name="bias",
                              layer=sm_layer,
                              inputs_data=(gnn_layer_output,),
                              gt_labels=labels,
                              eps=1e-4,
                              weight_decay=optim.weight_decay)
```

[29] ✓ 0.0s

Python

```
dW, db = sm_layer.backward_pass(optim, need_update=False)
```

[30] ✓ 0.0s

Python

We then get the gradients of Softmax layer.

```
dW, db = sm_layer.backward_pass(optim, need_update=False)
```

[31] ✓ 0.0s

Python

We now assert that the true and approximate gradients are very close to each other. If not, something went wrong in our implementation.

```
implementation_check(dW, dW_approx, db, db_approx)
```

[32] ✓ 0.0s

Python

... congrats, your implementation passes the test !!!

Great! The forward and backward passes for the **GNN_Layer** class have been implemented correctly, and the checks have passed successfully. The outputs from the forward pass match the expected results, confirming that the implementation is functioning as intended. Cursor did a good job.

The Cursor's implementation is a little different from the reference solution, but both implementations yield the same results, confirming their correctness.

5(f)

We are now ready to setup our classification network! Use the GNN and Softmax layers to setup the network.

input: complete Q6, setup the network following the reference

Cursor output:

```
#         first_gnn_layer = GNN_Layer(input_dim=??,
#                                     output_dim=??,
#                                     name='layer_0')
#
#         first_gnn_layer = GNN_Layer(input_dim=input_dim,
#                                     output_dim=hidden_dim[0],
#                                     name='layer_0')
#
#         self.layers.append(first_gnn_layer)
#
#         for layer in range(num_layers):
#             gnn_temp = GNN_Layer(input_dim=??,
#                                   output_dim=??,
#                                   name=f'layer_{layer}')
#             self.layers.append(gnn_temp)
#
#         for layer in range(1, num_layers):
#             gnn_temp = GNN_Layer(input_dim=hidden_dim[layer-1],
#                                   output_dim=hidden_dim[layer],
#                                   name=f'layer_{layer}')
#             self.layers.append(gnn_temp)
#
#         last_gnn_layer = Softmax_Layer(input_dim=??,
#                                         output_dim=??,
#                                         name='sm_layer')
#         self.layers.append(last_gnn_layer)
#
#         last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
#                                         output_dim=output_dim,
#                                         name='sm_layer')
#         self.layers.append(last_gnn_layer)
```

```
def embedding(self, A, X):
    H = X
    for layer in self.layers[:-1]:
        # H = ??
        pass
    # return
    H = layer.forward_pass(A, H)
    return H

def forward_pass(self, A, X):
    # H = ??
    H = X
    for layer in self.layers[:-1]:
        H = layer.forward_pass(A, H)
    out = self.layers[-1].forward_pass(H)
    pass
    # return
    return out
```

^ 5 of 6 v

Undo %N

Keep %Y

^ 4 / 6 v

Undo Cell %N

Keep Cell %Y

Solution:

```

first_gnn_layer = GNN_Layer(input_dim=input_dim,
                             output_dim=hidden_dim[0],
                             name='layer_0')

self.layers.append(first_gnn_layer)

for layer in range(num_layers):
    gnn_temp = GNN_Layer(input_dim=self.layers[-1].W.shape[0],
                          output_dim=hidden_dim[layer],
                          name=f'layer_{layer+1}')
    self.layers.append(gnn_temp)

last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
                                output_dim=output_dim,
                                name='sm_layer')
self.layers.append(last_gnn_layer)

def __repr__(self):
    return '\n'.join([str(layer) for layer in self.layers])

def embedding(self, A, X):
    H = X
    for layer in self.layers[:-1]:
        H = layer.forward_pass(A, H)
    return np.asarray(H)

def forward_pass(self, A, X):
    H = self.embedding(A, X)
    out = self.layers[-1].forward_pass(H)
    return np.asarray(out)

```

Python

There's something wrong here. The model from Cursor is 1 layer less than the reference solution. In Cursor's output, `input_dim=hidden_dim[layer-1]`, which causes the first layer to be skipped.

5(g)

Instantiate the GNN model with the correct input and output dimensions

5(h)

With the model, data and optimizer ready, fill in the todos in the training loop function and train your model. Plot the clustered data.

Cursor output:

```
for epoch in range(num_epochs):
    # TODO
    # y_pred = ??
    y_pred = model.forward_pass(A_hat, X)
    training_optim(y_pred, y_true, nodes_to_be_used_for_training)

    for layer in reversed[Any](model.layers):
        layer.backward_pass(training_optim, need_update=True)

    embeddings.append(model.embedding(A_hat, X))
    y_preds.append(y_pred)
    acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
        [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
    ]
    accs.append(np.mean(acc_temp))

    # loss_temp = ??
    loss_temp = get_cross_entropy(y_pred, y_true)
    # train_loss_temp = ??
    train_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_training])
    # test_loss_temp = ??
    test_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_testing])

    training_losses.append(train_loss_temp)
    testing_losses.append(test_loss_temp)

    if test_loss_temp < minimum_loss:
        minimum_loss = test_loss_temp
        ealy_stopping_counter = 0
    else:
        ealy_stopping_counter += 1

    if ealy_stopping_counter > ealy_stopping_steps:
        print("Training Stopped due to Early stopping!")
        break

    if epoch % 100 == 0:
        print(f"epoch #: {epoch+1} | train Loss: {train_loss_temp:.3f} | test Loss: {test_loss_temp:.3f}")
```


Solution:

```
for epoch in range(num_epochs):
    y_pred = model.forward_pass(A_hat, X)
    training_optim(y_pred, y_true, nodes_to_be_used_for_training)

    for layer in reversed(model.layers):
        layer.backward_pass(training_optim, need_update=True)

    embeddings.append(model.embedding(A_hat, X))
    y_preds.append(y_pred)
    acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1)) [
        [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
    ]
    accs.append(np.mean(acc_temp))

    loss_temp = get_cross_entropy(y_pred, y_true)
    train_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_training])
    test_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_testing])

    training_losses.append(train_loss_temp)
    testing_losses.append(test_loss_temp)

    if test_loss_temp < minimum_loss:
        minimum_loss = test_loss_temp
        ealy_stopping_counter = 0
    else:
        ealy_stopping_counter += 1

    if ealy_stopping_counter > ealy_stopping_steps:
        print("Training Stopped due to Early stopping!")
        break

    if epoch % 100 == 0:
        print(f"epoch #: {epoch+1} \t| train Loss: {train_loss_temp:.3f} \t| test Loss: {test_loss_temp:.3f}
        ")
```

The answer is completely correct. The training loop has been implemented properly, and the model has been trained successfully. The plotted clustered data shows the expected separation of the two groups in Zachary's Karate Club, indicating that the GNN model has learned to classify the nodes based on their features and connections in the graph.

6. Implementing Muon

In this assignment, you will implement key components of a Muon optimizer in PyTorch. For this assignment, we recommend using Google Colab as some PCs or laptops may not support GPU-based PyTorch. You will be implementing the code at [q_coding_muon.ipynb](#), then answering the questions in the notebook in your submission. The notebook itself does not need to be submitted.

Part 1: Implementing Newton-Schulz

The core of Muon is using matrix orthogonalization on a typical gradient update matrix. After orthogonalization:

- Singular values become more uniform
- Updates act across all directions in parameter space
- The neural network can utilize its full parameter capacity (because parameters are all receiving nontrivial gradients)

In Muon, matrix orthogonalization is done using Newton-Schulz. Newton-Schulz relies on iteration using an odd matrix polynomial. In this problem, we consider a very simple cubic polynomial, but many odd polynomials can be used (state-of-the-art implementations use specific, tuned quintic polynomial).

Complete the following code to implement Newton-Schulz, using the matrix polynomial in the comments.

input: complete Part 1, implement Newton-Schulz following the reference

Cursor output:

```
#####
# TODO: YOUR CODE HERE
#####
# Apply Newton-Schulz iterations:  $f(X) = (3X - X^3)/2$ 
for _ in range(num_iters):
    # Compute  $X^3 = X @ X^T @ X$ 
    X_cubed = X @ X.mT @ X
    # Apply polynomial:  $(3X - X^3)/2$ 
    X = (3 * X - X_cubed) / 2
#####
```

Solution:

```
# Apply Newton-Schulz iterations using the cubic polynomial
for _ in range(num_iters):
    X = (3 * X - torch.matmul(torch.matmul(X, X.mT), X)) / 2
```

The Newton-Schulz implementation is correct. The function successfully computes the orthogonalized matrix using the specified cubic polynomial iteration.

Part 2: Implementing Muon Update

Now, we implement the update in a Muon optimizer. Given parameter matrix W with momentum matrix M , the pseudocode for the Muon update proceeds as follows:

```
d_out, d_in = M.shape

# Apply Newton-Schulz orthogonalization
M ← newton_schulz_orthogonalize(M, ns_iters)

# Apply muP scaling factor for consistent update magnitude
M ← M · sqrt(max(1, d_out / d_in))
```

Then, the Muon update is used later to update the parameters W :

```
# Update the parameter matrix
W ← W - lr * M
```

Complete the following code to implement the Muon update following the above pseudocode.

input:

complete Part 2, implement Muon update following the reference

Cursor output:

```
def muon_update(grad, momentum, beta=0.95, ns_iters=5):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) * grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
        reshaped = True
        update = update.view(update.size(0), -1)

    #####
    # TODO: YOUR CODE HERE
    #####

    # Apply Newton-Schulz orthogonalization
    update = newton_schulz_orthogonalize(update, ns_iters)

    # Apply muP scaling factor for consistent update magnitude
    d_out, d_in = update.size(-2), update.size(-1)
    update.mul_(max(1, d_out / d_in)**0.5)

    #####
    # Restore shape if needed
    if reshaped:
        update = update.view(original_shape)

    return update
```

Solution:

```
def muon_update(grad, momentum, beta=0.95, ns_iters=5):
    momentum.lerp_(grad, 1 - beta) # momentum = beta * momentum + (1-beta) * grad
    update = momentum.clone()

    # If the parameter is a convolutional kernel, then flatten to a 2D matrix
    original_shape = update.shape
    reshaped = False
    if update.ndim > 2:
        reshaped = True
        update = update.view(update.size(0), -1)

    # Apply orthogonalization
    update = newton_schulz_orthogonalize(update, num_iters=ns_iters)

    # Apply muP scaling
    update.mul_(max(1, update.size(-2) / update.size(-1))**0.5)

    # Restore shape if needed
    if reshaped:
        update = update.view(original_shape)

    return update
```

The Muon update implementation is correct. The function successfully applies the Newton-Schulz orthogonalization to the momentum matrix and scales it according to the specified muP scaling factor before updating the parameter matrix W.